

Public Library Management System

Project Planning & Design Documentation

1. Project Overview

The Public Library Management System is a web-based application that allows users to browse books and view real-time availability. The system also provides an administrative interface for managing book inventory and handling issue and return operations.

The primary goal of the project is to ensure accurate inventory management using real-time updates while keeping the system simple, scalable, and user-friendly.

2. User Roles

- **Public User**
 - Browse available books
 - View book details
 - Check real-time availability
- **Admin**
 - Add, update, and delete books
 - Issue and return books
 - Manage inventory

3. Core Features

- Public book catalog
- Real-time availability updates
- Admin inventory management
- Issue and return operations
- Secure admin authentication

4. Out of Scope (Phase 1)

The following features are intentionally excluded from the initial phase:

- Trending analytics
- Artificial Intelligence or Machine Learning features
- Public user authentication
- Book recommendations

5. Technology Stack

- **Frontend:** React
- **Backend:** Node.js, Express
- **Database:** MongoDB
- **Real-Time Communication:** Socket.IO
- **Authentication:** JWT (Admin only)

6. Development Approach

The project follows an API-driven development approach. Backend services are implemented first to define business logic and data consistency, followed by frontend development that consumes these APIs.

7. System Architecture

7.1. High-Level Architecture

React Frontend

```
|-- REST API Calls
|-- Socket.IO Client
|
```

Node.js + Express Backend

```
|-- Business Logic
|-- REST APIs
|-- Socket.IO Server
|
```

MongoDB Database

7.2. Architecture Principles

- Backend acts as the single source of truth
- Frontend handles presentation only
- Real-time events notify clients of inventory changes
- REST APIs handle data operations

8. Real-Time Update Flow

1. Admin issues or returns a book
2. Backend updates the database
3. Backend emits a Socket.IO event
4. All connected clients receive updated availability instantly

9. Data Models

9.1. Book Collection

- Title: String
- Author: String
- ISBN: String
- Category: String
- Total Copies: Number
- Available Copies: Number
- Description: String
- Created At: Date
- Updated At: Date

9.2. Admin Collection

- Name: String
- Email: String
- Password Hash: String
- Role: Admin

9.3. Design Decisions

- Availability is stored for fast read operations
- Inventory updates are atomic
- No public user collection is maintained

10. API Plan

10.1. Public APIs

- **GET /api/books** – Retrieve all books
- **GET /api/books/:id** – Retrieve book details

10.2. Admin APIs (Authenticated)

- **POST /api/admin/login** – Admin authentication
- **POST /api/books** – Add a new book
- **PUT /api/books/:id** – Update book details
- **DELETE /api/books/:id** – Delete a book
- **POST /api/books/:id/issue** – Issue a book
- **POST /api/books/:id/return** – Return a book

10.3. Real-Time Events

- **Event Name:** availabilityUpdated
- **Payload:**

```
{  
  bookId: string,  
  availableCopies: number  
}
```

11. Task Breakdown

11.1. Backend Tasks

Project setup

Database connection

Book schema

CRUD APIs

Issue/return logic

Socket.IO integration

11.2. Frontend Tasks

Project setup

Public catalog page

Book details page

Admin dashboard

Socket.IO listener

11.3. Finalization

Error handling

Loading states

Documentation update

Demo preparation

12. Conclusion

This document outlines the complete planning and design phase of the Public Library Management System. By following a structured and API-driven approach, the project aims to ensure scalability, maintainability, and real-time accuracy.